DHBW Heilbronn

Software Engineering II.1 – Block 1

Software Engineering II| 29.09.2026 |
Dr. rer. nat. Ana Nicolaescu

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7
4 UE	4 UE	4 UE	4 UE	4 UE	4 UE	4 UE
Einführung & Vorgehensmodelle	Requirements Engineering	Entwurf & Entwurfsmuster	Implementierung & Versionsmanagement	Testen (Blackbox)	Whitebox, Wartung & Change Request	Management & Abschlusspräsentation

Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository

Teams, Repo, Vorgehen	REQUIREMENTS.md Use Cases · AK · Prototyp	ARCHITECTURE.md Klassen- · Sequenzdiagramm	Build · Git · PR Review · CI grün	TESTING.md Unit/Integr. · Blackbox	Coverage · Refactoring Change Request	RISK_ANALYSIS.md Präsentation

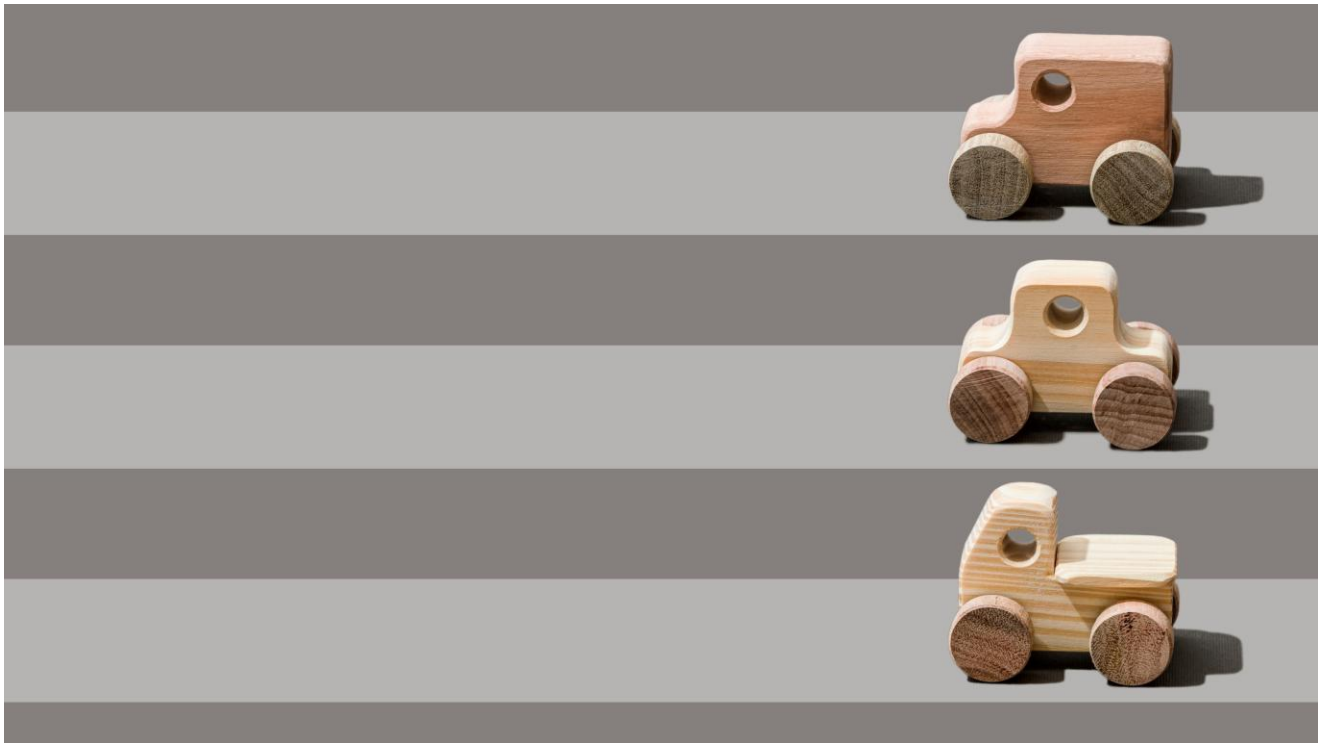
Warum braucht man Vorgehensmodelle?

- Komplexitätsbeherrschung
  - Softwareprojekte sind heute zu groß für „one man shows“
  - Strukturierung des Lebenszyklus
- Disziplin und Verbindlichkeit: Rollen, Aktivitäten, Artefakte
  - Ergebnisse: Es wird festgelegt, was am Ende einer Phase vorliegen muss (z.B. ein Lastenheft oder ein Architekturdiagramm).
  - Verlässlichkeit**:** Besonders in regulierten Branchen (Medizintechnik, Luftfahrt) ist ein Wasserfall-orientierter Prozess oft gesetzlich oder durch Normen vorgeschrieben.
- Monitoring: Meilensteine + Transparenz

Warum braucht man Vorgehensmodelle?

- Risikomanagement durch Vorhersehbarkeit
  - Immer wieder die Gleichen Aktivitäten in der gleichen Rheinfolge
- Standardisierung und Skalierbarkeit
  - Mitarbeiter können zwischen Projekte wechseln
  - „Wenn man den Prozess kennt, ist man gleich eingearbeitet“

Lass uns mit einem Experiment starten



Stimmen Sie zu?

Bitte kurz argumentieren

„Je mehr wir zu Beginn analysieren, desto besser wird das entwickelte System“

„Es ist besser, den Kunden ein fertiges Produkt zu zeigen als Iterationen davon“

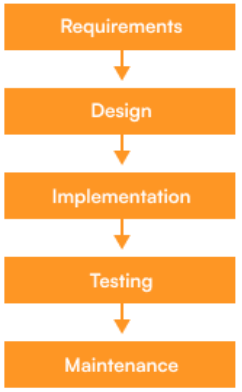
Agile vs. Waterfall: Kurze Übersicht

Waterfall

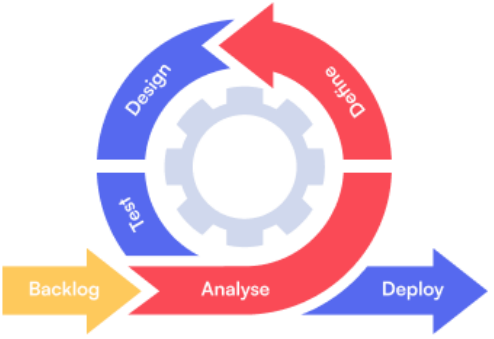
- Linearer und sequenzieller Ansatz
- Es wird in festen Phasen gearbeitet

Agile

- Iterative und inkrementell
- Ermöglich es Teams, flexibler auf Änderungen zu reagieren
- Feedback wird kontinuierlich integriert.



Waterfall methodology



Agile methodology

Photo Management Challenge Teil 1

Schritt 1: Gruppenbildung

- Gruppe 1: Auftraggeber; Gruppe 2: Auftragnehmer; Alle anderen: aktive Beobachter

- Die erste Gruppe überlegt sich einen Auftrag für ein App für die Erstellung und Verwaltung von Photo Alben. Die Zielgruppe sollten Teenagers sein
    - Diese Gruppe soll sich bewusst manche Anforderungen ausdenken die sehr explizit kommuniziert werden müssen
    - Es gibt aber auch Anforderungen, die Ihnen sehr wichtig sind (z.B. Integration mit der DM App zum Ausdrucken von Alben, Skalierbarkeit, etc.), die sie nur dann kommunizieren, wenn die zweite Gruppe explizit danach fragt
    - Sie haben 15 Minuten Zeit, um eine Vision vorzubereiten. In dieser Zeit überlegen sich die anderen Gruppen, welche Fragen gestellt werden müssen
- Die zweite Gruppe ist der Auftragnehmer. Sie haben 15 Minuten, um Anforderungen zu sammeln

Photo Management Challenge Teil 1

Schritt 2:

- Gruppe 2 sammelt Anforderungen von Gruppe 1
- Alle anderen: notieren, welche Fragen sie gehabt hätten, die Gruppe 2 nicht gestellt hat
- Im Anschluss:
  - Basierend *nur* auf die von Gruppe 2 erstellten Anforderungen, machen alle Gruppen Prototypen
  - Nur Gruppe 1, erstellt ein Prototyp, die auch ihre „implizite/nicht entdeckte“ Anforderungen beinhaltet

Photo Management Challenge Teil 1

Schritt 3:

- 2 Gruppen präsentieren Ihr Prototyp
- Gruppe 1 äußert sich dazu: ist es das, was sie sich vorgestellt haben? Würden sie das so annehmen?

Photo Management Challenge Teil 2

Schritt 4:

- Gruppe 1 gibt Feedback zu dem Prototyp
- Die anderen Gruppen haben Zeit, das Prototyp zu bearbeiten

} 1 Mal wiederholen

Schritt 5:

- 2 Gruppen präsentieren Ihr Prototyp
- Gruppe 1 äußert sich dazu und zeigt auch ihr Prototyp: ist es das, was sie sich vorgestellt haben? Würden sie das so annehmen?
  - Haben die Iterationen geholfen?
  - Wie war denn die Erfahrung?

Stimmen Sie (noch) zu?

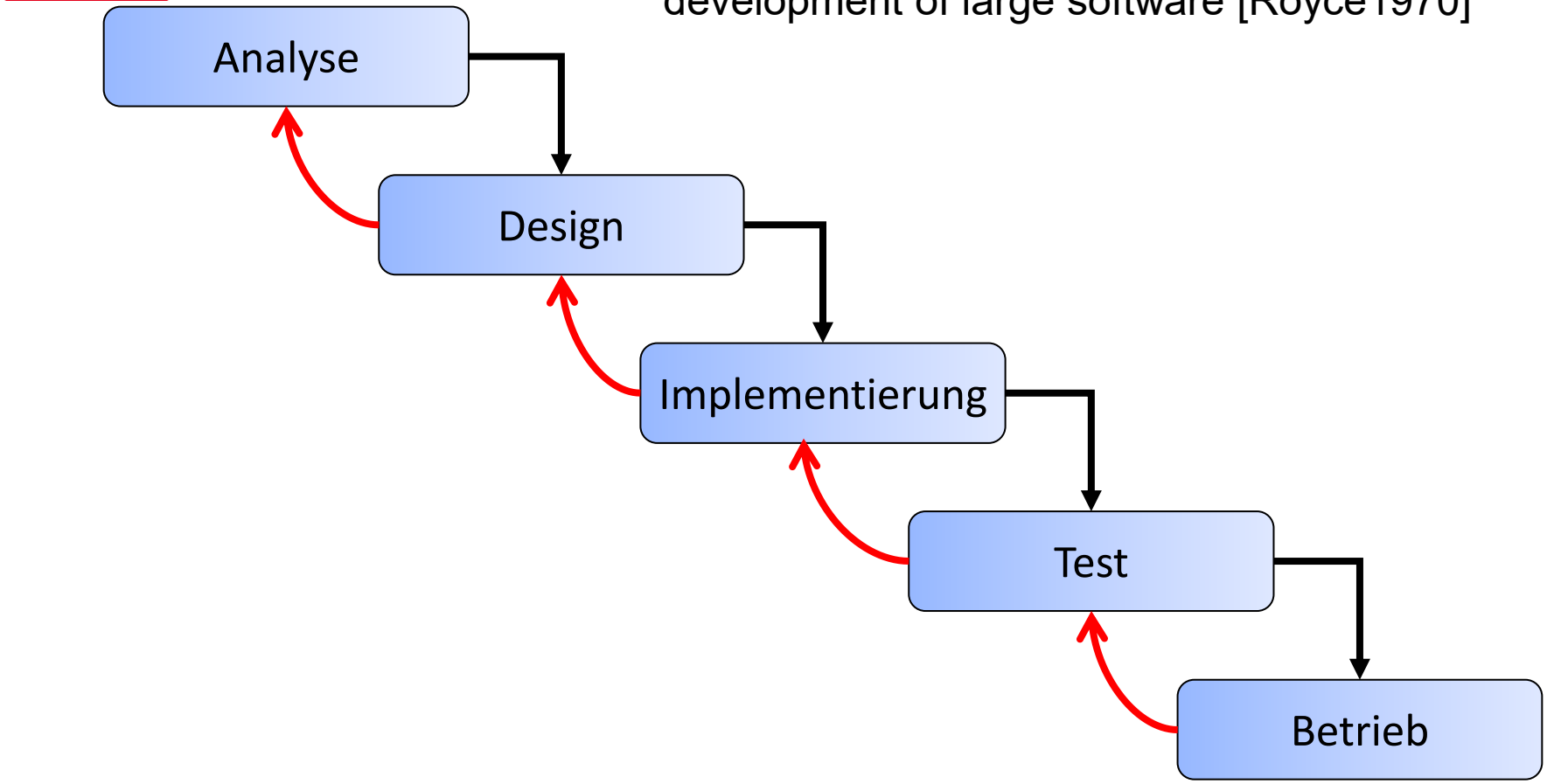
Bitte kurz argumentieren

„Je mehr wir zu Beginn analysieren, desto besser wird das entwickelte System“

„Es ist besser, den Kunden ein fertiges Produkt zu zeigen als Iterationen davon“

Waterfall

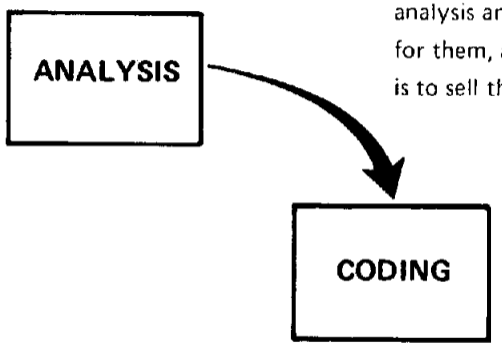
1970, Winston W. Royce: Managing the development of large software [Royce1970]



Ausflug: Zurück in die 70er

INTRODUCTION

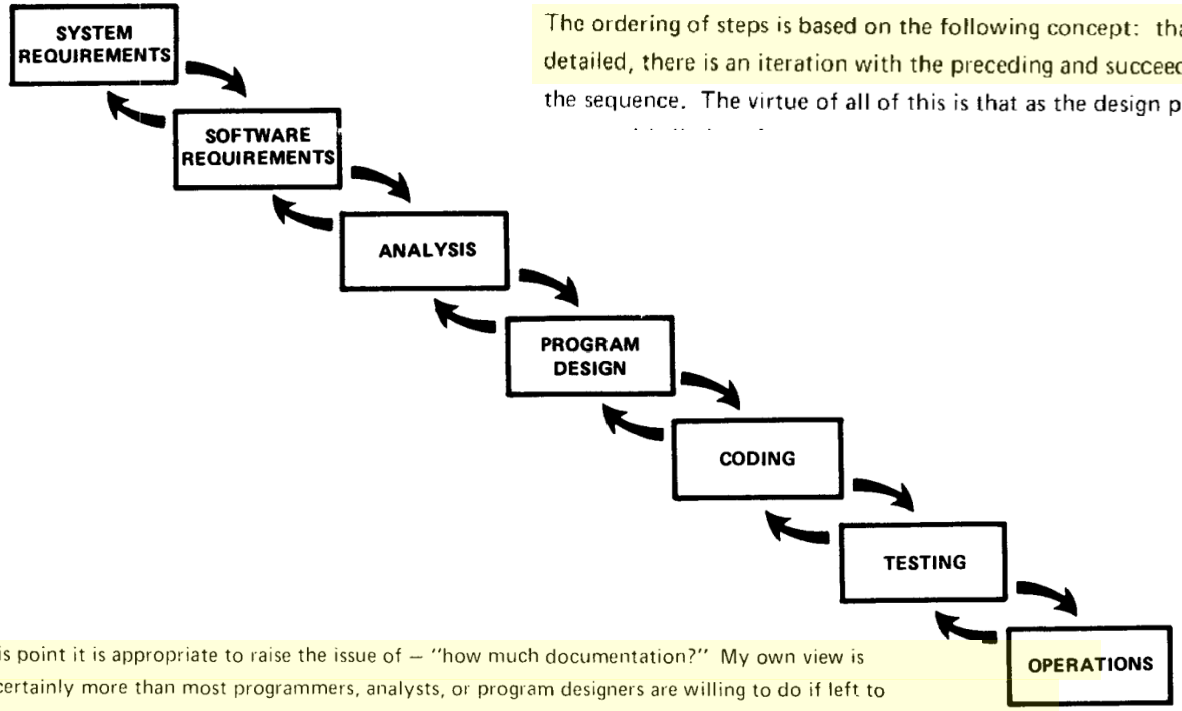
I am going to describe my personal views about managing large software developments. I have had various assignments during the past nine years, mostly concerned with the development of software packages for spacecraft mission planning, commanding and post-flight analysis. In these assignments I have experienced different degrees of success with respect to arriving at an operational state, on-time, and within costs. I have become prejudiced by my experiences and I am going to relate some of these prejudices in this presentation.



to failure. Many additional development steps are required, none contribute as directly to the final product as analysis and coding, and all drive up the development costs. Customer personnel typically would rather not pay for them, and development personnel would rather not implement them. The prime function of management is to sell these concepts to both groups and then enforce compliance on the part of development personnel.

Figure 1. Implementation steps to deliver a small computer program for internal operations.

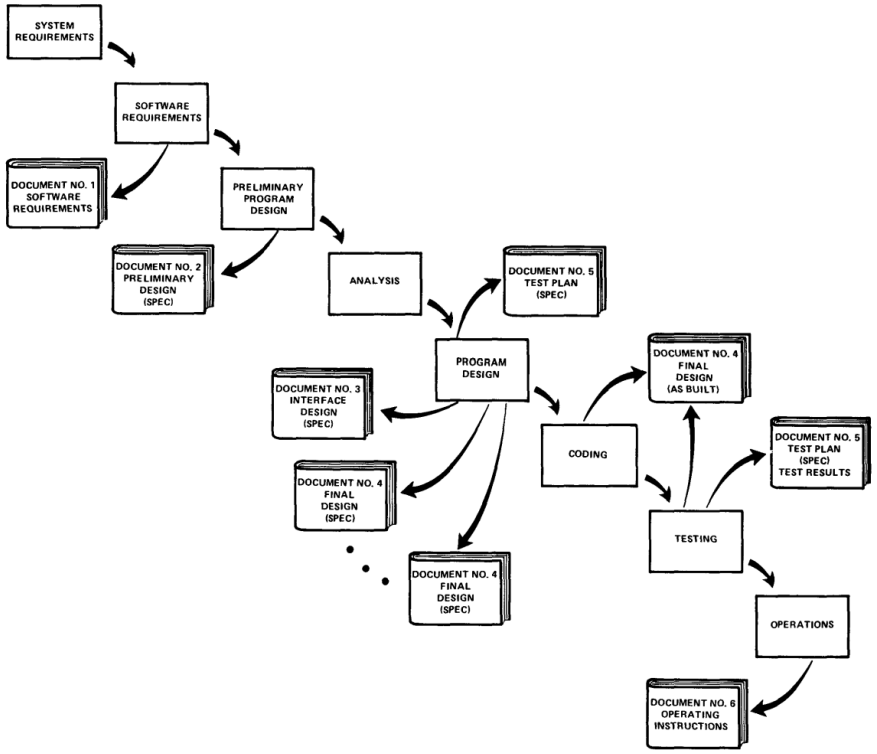
Ausflug: Zurück in die 70er



The ordering of steps is based on the following concept: that as each step progresses and the design is further detailed, there is an iteration with the preceding and succeeding steps but rarely with the more remote steps in the sequence. The virtue of all of this is that as the design proceeds the change process is scoped down to

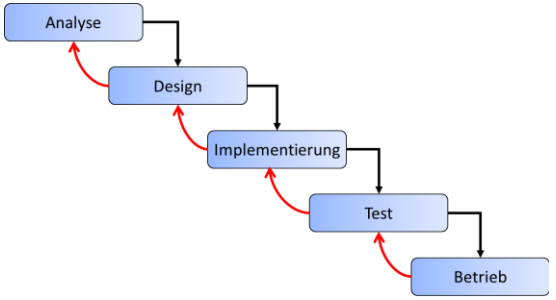
At this point it is appropriate to raise the issue of – "how much documentation?" My own view is "quite a lot;" certainly more than most programmers, analysts, or program designers are willing to do if left to their own devices. The first rule of managing software development is ruthless enforcement of documentation requirements.

Ausflug: Zurück in die 70er



Klare Erwartungen bezüglich des Outcomes jeder einzelnen Phase.

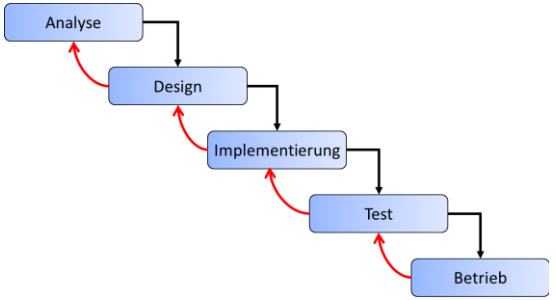
Waterfall – Vorteile?



- Disziplin und Struktur
- Planbarkeit
- Einfachheit
- Qualitätskontrolle
- Stabilität

Waterfall – Nachteile?

- Das "Late-Disaster"-Risiko
  - Eine Korrektur am Ende ist extrem teuer.
- Unflexible Anforderungen
- Kein Prototyping
  - Das Risiko, "am Kunden vorbei" zu entwickeln, ist hoch.
- Phasenblockaden
- Dokumentationslastig



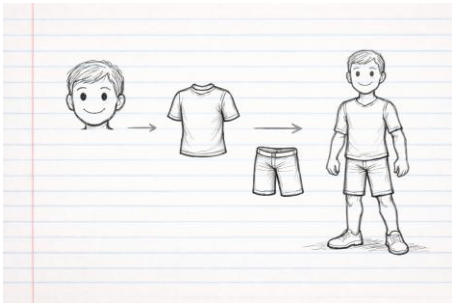


Waterfall – Wo könnte es noch passen?

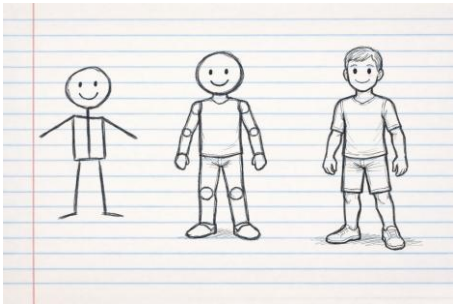
- Kurze, einfache Projekte
- Wettbewerbe mit fixen Anforderungen
- Kritische Systeme (Safety-Critical)
  - Medizintechnik
  - Luft- und Raumfahrt
  - Kerntechnik etc.

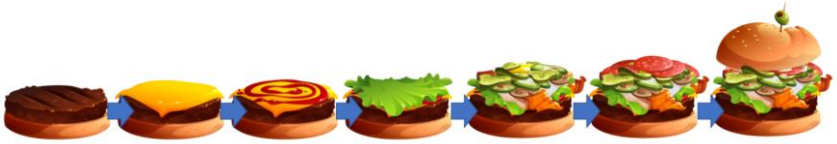
Inkrementelle vs. Iterative Entwicklung

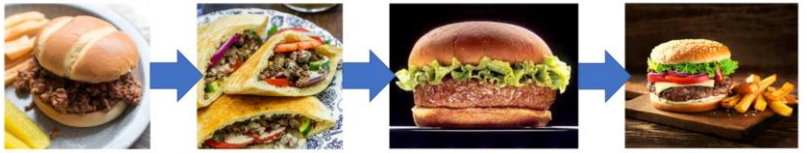
Inkrementell
- Stück für Stück



Iterativ
- Das gleiche, immer besser





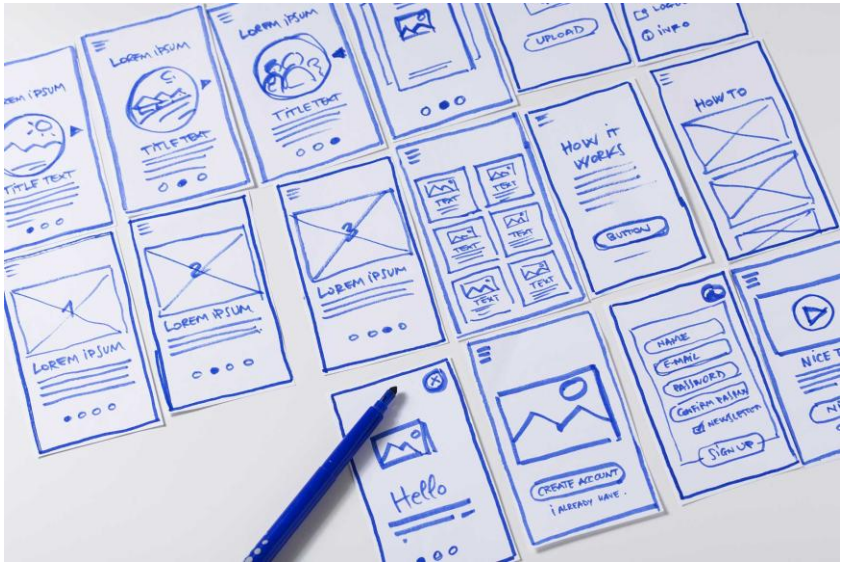


[5dvision]

Prototypentwicklung: Was ist ein Prototyp?

- Statt Monate auf ein fertiges Produkt zu warten, erstellt man eine vorläufige, vereinfachte Version der Software.

- Wozu nützlich?
  - Anforderungsklärung
  - Machbarkeitsnachweis
  - Frühes Feedback
  - Akzeptanz

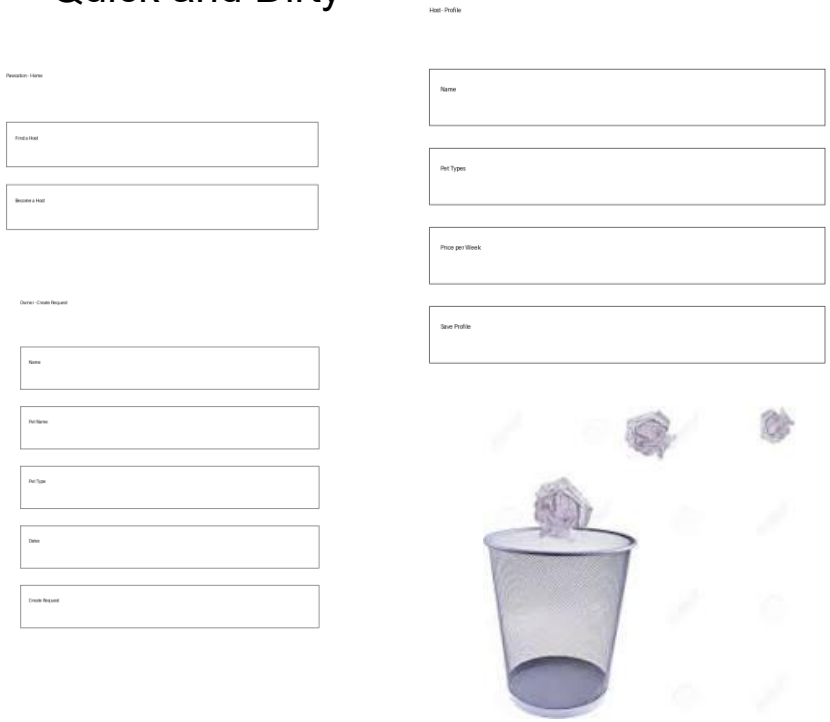


[RockPrototype]

Wegwerf- vs. Evolutionäres Prototyping

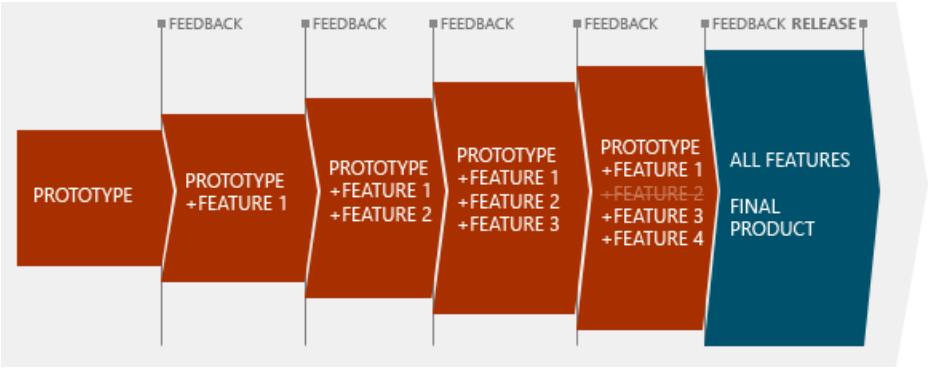
Wegwerfprototyping

- Quick and Dirty



Evolutionäres Prototyping

- Ausbauen statt Wegwerfen



Clickable Prototypes

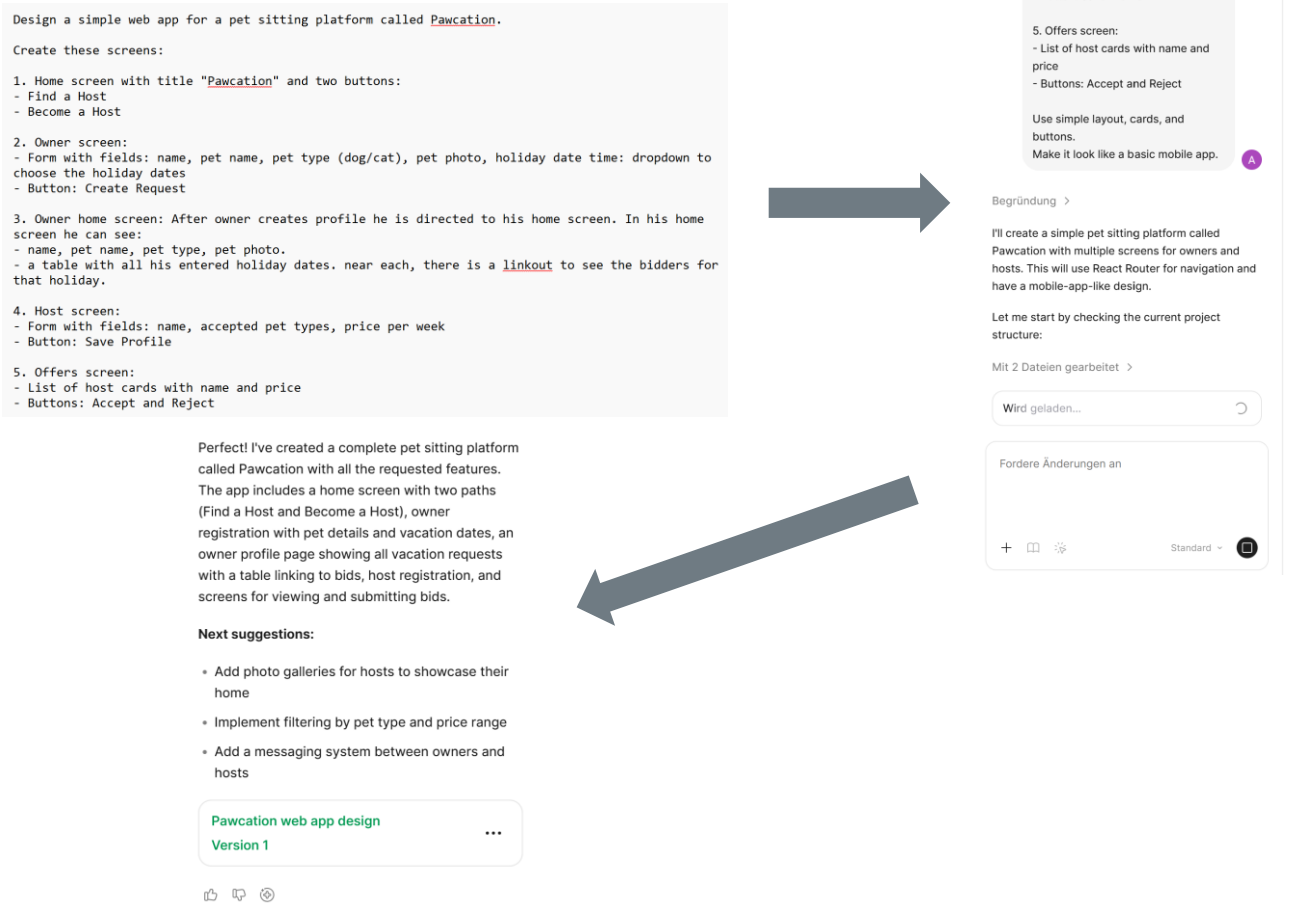
Figma

ProtoPie

UXPin

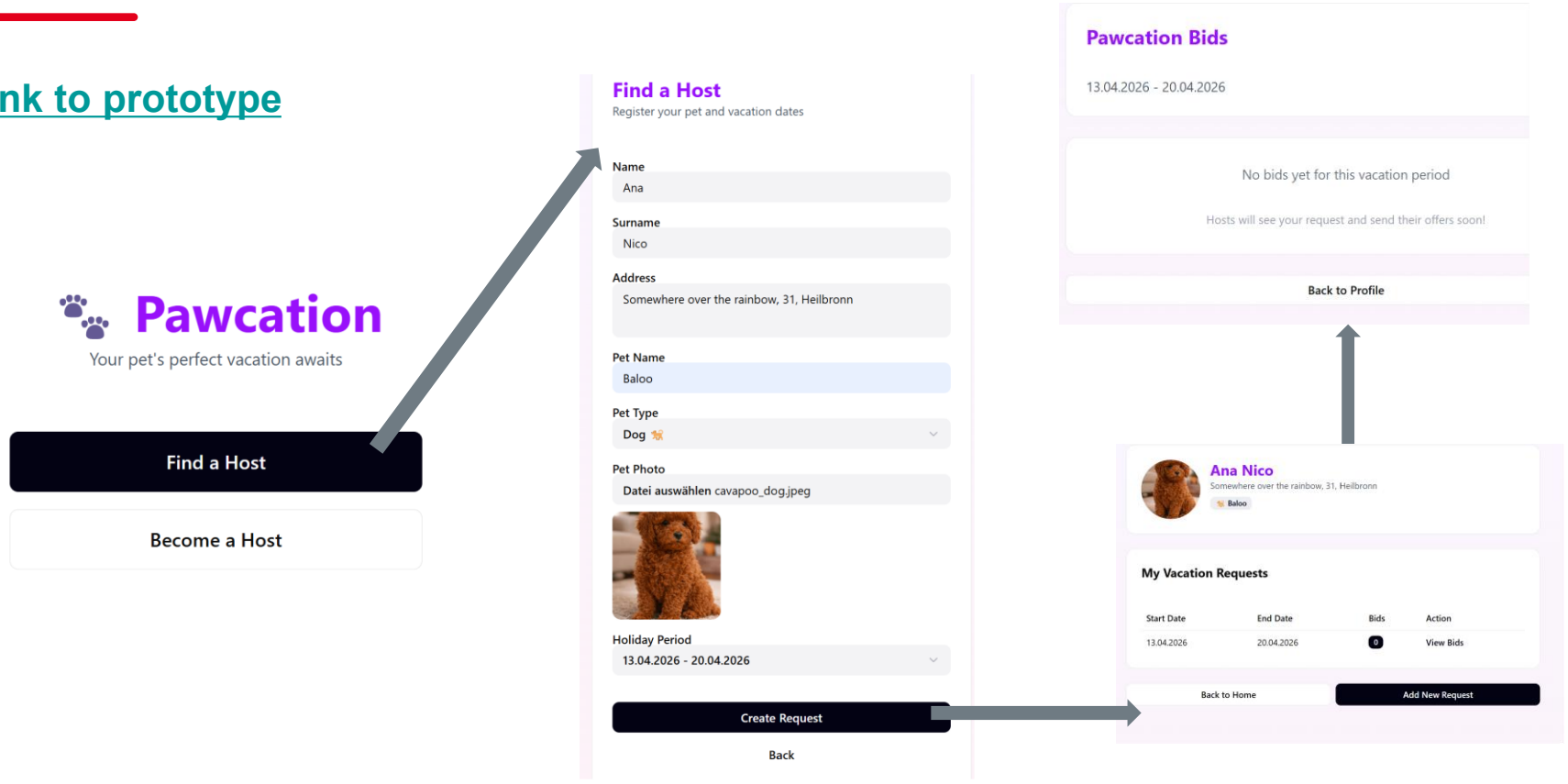
Marvel

Uizard



Figma: Mein generiertes Prototyp

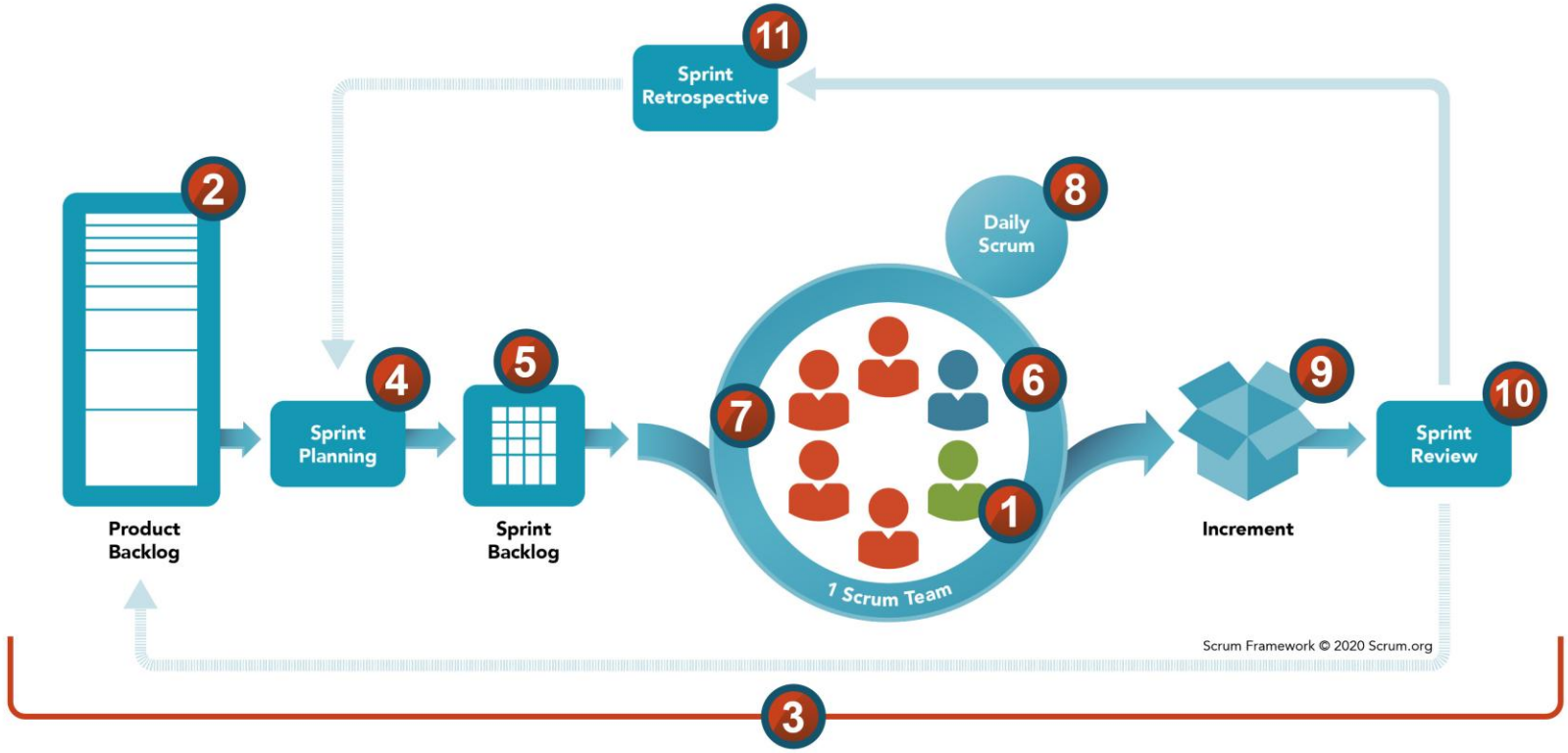
[Link to prototype]



2001: Das Agile Manifest



Der Scrum Kreislauf



Quiz: Scrum

- Wer ist in Scrum primär dafür verantwortlich, den Wert des Produkts zu maximieren und das Product Backlog zu verwalten?
  - Scrum Master
  - Developer
  - Stakeholder
  - Product Owner

Quiz: Scrum

- Wer ist in Scrum primär dafür verantwortlich, den Wert des Produkts zu maximieren und das Product Backlog zu verwalten?
  - Scrum Master
  - Developer
  - Stakeholder
  - **Product Owner**

Quiz: Scrum

- Was passiert in Scrum mit Product-Backlog-Einträgen, die am Ende eines Sprints nicht abgeschlossen wurden?
  - Sie wandern zurück in das Product Backlog
  - Sie werden automatisch im nächsten Sprint übernommen
  - Sie werden als Teil des Inkrementes gewertet, wenn sie fast richtig sind
  - Das Team muss Überstunden machen, um den Sprint zu verlängern

Quiz: Scrum

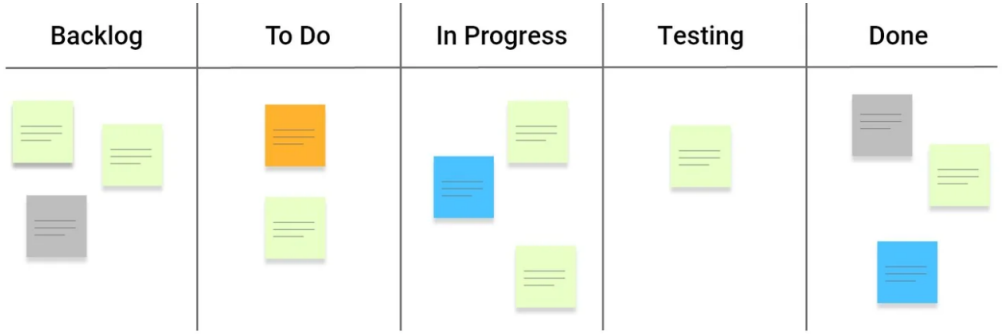
- Was passiert in Scrum mit Product-Backlog-Einträgen, die am Ende eines Sprints nicht abgeschlossen wurden?
  - **Sie wandern zurück in das Product Backlog**
  - Sie werden automatisch im nächsten Sprint übernommen
  - Sie werden als Teil des Inkrementes gewertet, wenn sie fast richtig sind
  - Das Team muss Überstunden machen, um den Sprint zu verlängern

Kanban

David Meyer – hat Multitasking studiert

Der Zeigarnick Effekt – Mentalles Overload durch nicht abeschlossene Arbeit

- Stop starting, start finishing
- Methode zur Visualisierung und Optimierung des Arbeitsflusses



改善

Kai = Change Zen = Good

Kanban Praktiken

•Visualisiere den Workflow

•Begrenze die parallele Arbeit (WIP – Work in Progress)

•Manage den Flow
- Reduziere Cycle-Time

•Mache Prozessregeln explizit

•Implementiere Feedbackschleifen
- Kanban Cadences (Meetings)

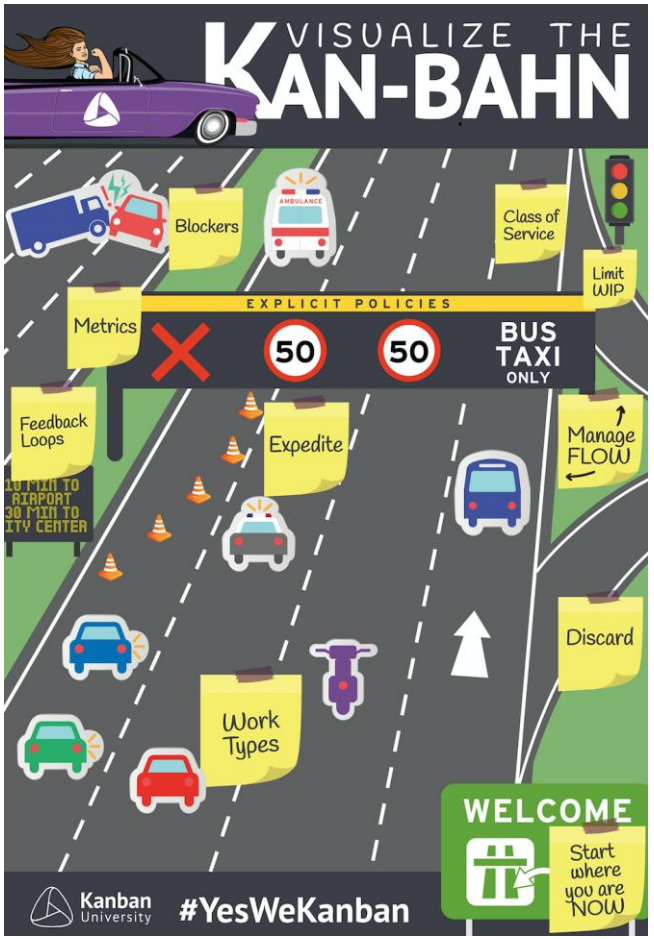
•Kontinuierliche Verbesserung (Kaizen)

Kanban Prinzipien



- Beginne dort, wo du jetzt bist
- Inkrementelle, evolutionäre Veränderung
  - „jeden Tag ein bisschen besser“
  - Kein Big Bang!
- Führung auf allen Ebenen

Kanban Gesamtoverview



https://kanban.university/kanban-guide/#kan-bahn

Quiz: Scrum vs. Kanban

- Was ist der grundlegende Unterschied in der Arbeitsplanung?
  - In Scrum gibt es keine Planung, in Kanban wird täglich geplant.
  - Scrum arbeitet in festen Zeitintervallen (Sprints), während Kanban auf einen kontinuierlichen Arbeitsfluss setzt.
  - In Kanban sind Sprints immer doppelt so lang wie in Scrum.
  - Scrum erlaubt jederzeit neue Aufgaben im laufenden Zyklus, Kanban verbietet das.

Quiz: Scrum vs. Kanban

- Was ist der grundlegende Unterschied in der Arbeitsplanung?
  - In Scrum gibt es keine Planung, in Kanban wird täglich geplant.
  - **Scrum arbeitet in festen Zeitintervallen (Sprints), während Kanban auf einen kontinuierlichen Arbeitsfluss setzt.**
  - In Kanban sind Sprints immer doppelt so lang wie in Scrum.
  - Scrum erlaubt jederzeit neue Aufgaben im laufenden Zyklus, Kanban verbietet das.

Quiz: Scrum vs. Kanban

- Welche Aussage zu den Rollen ist korrekt?
  - Kanban schreibt die Rollen Service Request Manager und Service Delivery Manager zwingend vor.
  - Scrum hat drei definierte Rollen (PO, Scrum Master, Developer); Kanban schreibt keine spezifischen Rollen vor.
  - Beide Modelle benötigen zwingend einen Projektleiter.
  - In Kanban übernimmt der Kunde die Rolle des Scrum Masters.

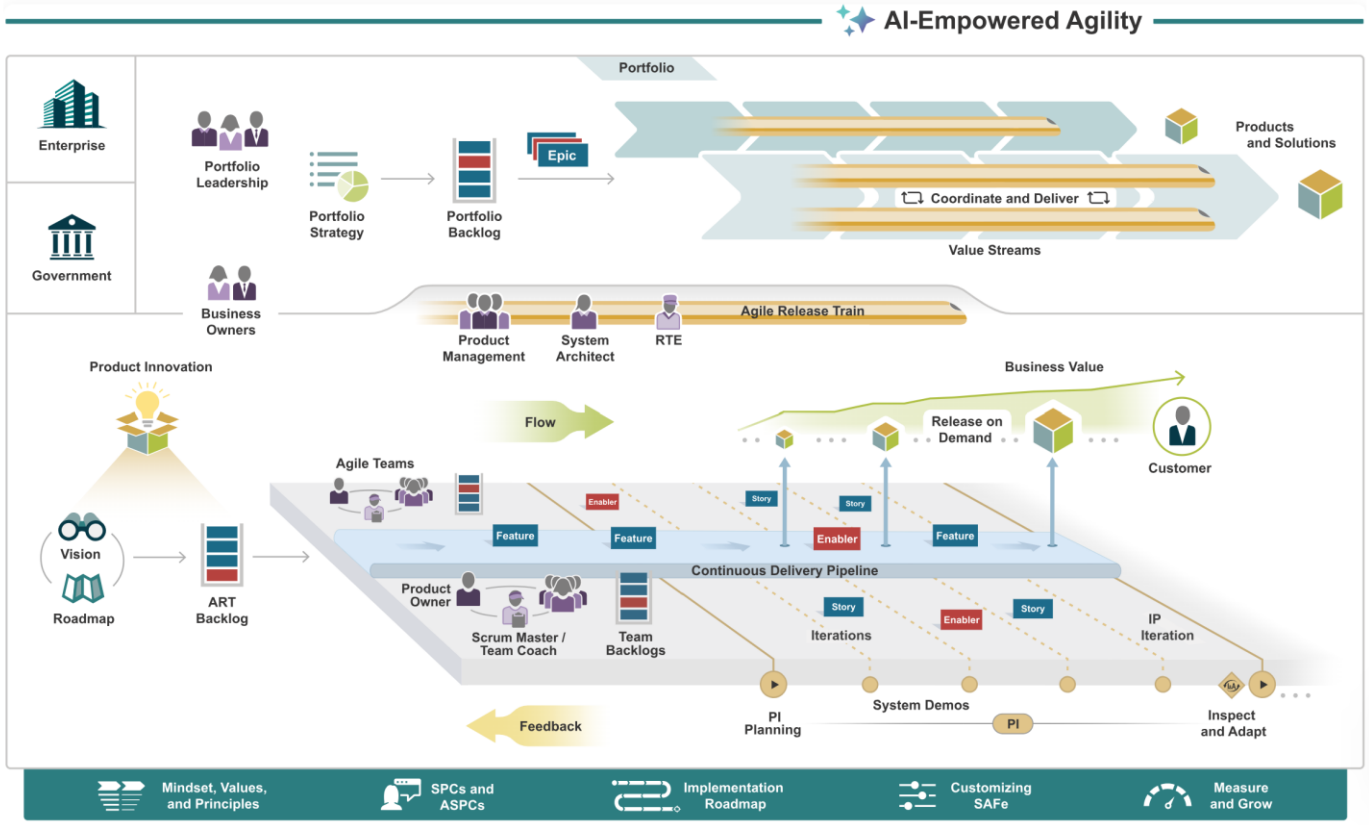
Scrum of Scrums (SoS)

- Was passiert, wenn viele Menschen (>10 Personen) agil zusammenarbeiten müssen?
  - In kleinere Teams aufteilen und koordinieren
- Scaled Agile
  - Unterstützt die Bereitstellung komplexer Lösungen
  - Mehrere agile Teams miteinander verbinden, die ein gemeinsames Ziel haben und sich koordinieren müssen.



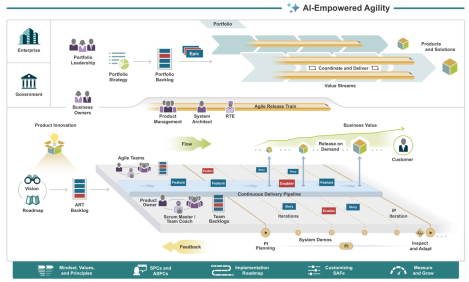
[SoS]

SAFe


AI-Empowered Agility
Portfolio
Enterprise
Government
Portfolio Leadership
Portfolio Strategy
Portfolio Backlog
Epic
Products and Solutions
Coordinate and Deliver
Value Streams
Business Owners
Agile Release Train
Product Management
System Architect
RTE
Product Innovation
Business Value
Flow
Release on Demand
Customer
Agile Teams
Vision
Roadmap
ART Backlog
Feature
Enabler
Story
Continuous Delivery Pipeline
Product Owner
Scrum Master / Team Coach
Team Backlogs
Iterations
IP Iteration
PI Planning
System Demos
PI
Inspect and Adapt
Feedback
Mindset, Values, and Principles
SPCs and ASPCs
Implementation Roadmap
Customizing SAFe
Measure and Grow

SAFe

SAFe wird in mehr als 100 Ländern eingesetzt, von über 1 Million zertifizierten SAFe-Praktikern.

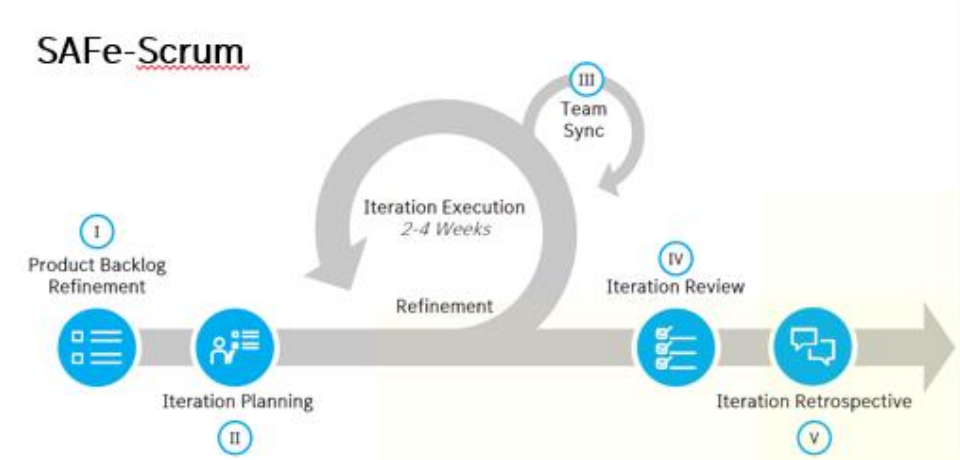


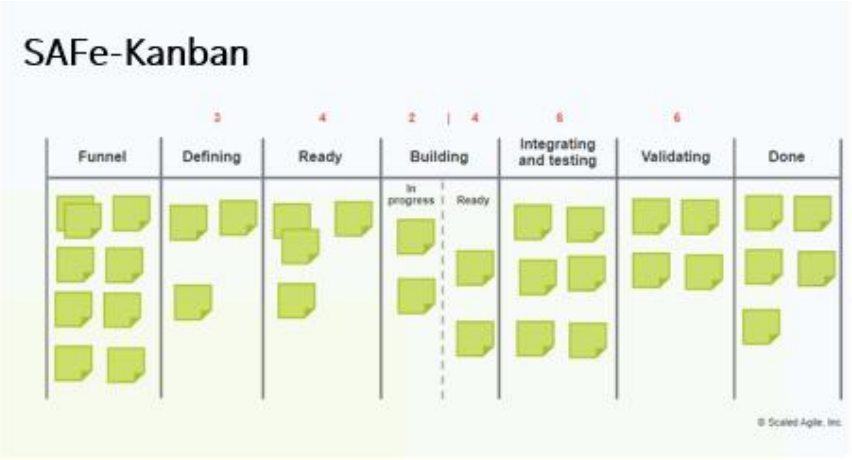
SAFe is also a huge Business Model – more than 500 Transformation and Platform Partners are supporting/ do trainings for SAFe



SAFe ist die am häufigsten verwendete skalierte agile Methode – sie wird von mehr als 37 % aller Unternehmen eingesetzt.

SAFe – kann mit Scrum & Kanban kombiniert werden

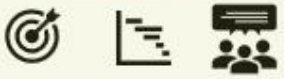




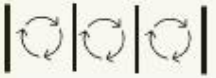
Adjustments in SAFE to Scrum & Kanban



PI-Planning results and some sync meetings



Common cadences: PI and Iterations



Vorgehensmodelle - Fazit

Aspekt	Plandriven (z.B. Wasserfall)	Change-driven (z.B. Scrum)
Fokus		
Anforderungen		
Risikosteuerung		
Dokumentation		

Vorgehensmodelle - Fazit

Aspekt	Plandriven (z.B. Wasserfall)	Change-driven (z.B. Scrum)
Fokus	Prozessdisziplin & Vorhersagbarkeit	Flexibilität & Kundennutzen
Anforderungen		
Risikosteuerung		
Dokumentation		

Vorgehensmodelle - Fazit

Aspekt	Plandriven (z.B. Wasserfall)	Change-driven (z.B. Scrum)
Fokus	Prozessdisziplin & Vorhersagbarkeit	Flexibilität & Kundennutzen
Anforderungen	Zu Beginn stabil und klar definiert	Entwickeln sich während des Prozesses
Risikosteuerung		
Dokumentation		

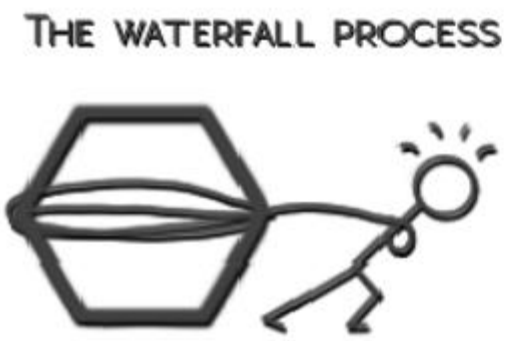
Vorgehensmodelle - Fazit

Aspekt	Plandriven (z.B. Wasserfall)	Change-driven (z.B. Scrum)
Fokus	Prozessdisziplin & Vorhersagbarkeit	Flexibilität & Kundennutzen
Anforderungen	Zu Beginn stabil und klar definiert	Entwickeln sich während des Prozesses
Risikosteuerung	Durch detaillierte Planung & Reviews	Durch kurze Feedbackschleifen
Dokumentation		

Vorgehensmodelle - Fazit

Aspekt	Plandriven (z.B. Wasserfall)	Change-driven (z.B. Scrum)
Fokus	Prozessdisziplin & Vorhersagbarkeit	Flexibilität & Kundennutzen
Anforderungen	Zu Beginn stabil und klar definiert	Entwickeln sich während des Prozesses
Risikosteuerung	Durch detaillierte Planung & Reviews	Durch kurze Feedbackschleifen
Dokumentation	Umfangreich und formell	"Just enough", Fokus auf Code

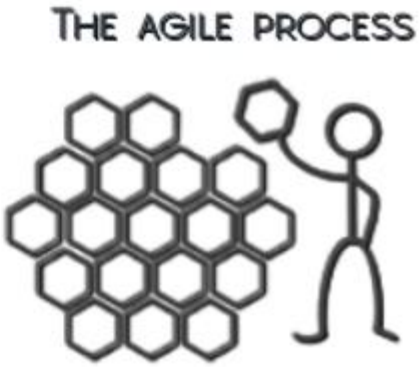
Vorgehensmodelle

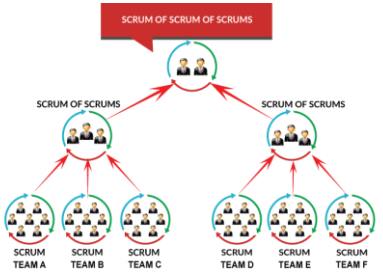


[scrum.org]









Vorgehensmodelle – wie funktionieren sie mit AI?

- Erfahrungen von Organisationen
- Eure Erfahrungen
- Skills
- Brauchen wir noch Prozesse?
- Herausforderungen
- Welche Prozesse sind passend für Agenten?

Referenzen

[Royce70] Winston W. Royce: Managing the development of large software

[Meadows97] Leverage Points: Places to Intervene in a System

[WikiVModell] Das V-Modell

[Boehm1979] Barry Boehm: Guidelines for Verifying and Validating Software Requirements and Design Specifications

[Boehm1979] Barry Boehm: A spiral model of software development and enhancement

[scrum.org] What is Iterative, Incremental Delivery? The Hunt for the Perfect Example

[5dvision] Incremental vs. Iterative

[RockPrototype] About Prototyping

[Nonaka1986] Takeuchi und Nonaka: The New New Product Development Game

[Sutherland1995] Jeff Sutherland und Ken Schwabe: Scrum Development Process

[SoS] https://www.agilest.org/scaled-agile/scrum-of-scrums/

[ScaledAgile] https://framework.scaledagile.com/#big-picture